TECHNICAL PROJECT DESCRIPTION

Encoderless AGV Localization and SLAM

ROS 2 Jazzy • Laser odometry • Inertial fusion • Raspberry Pi 5

Dakshitha (Dak) Amarasinghe | Robotic Systems Lab, Santa Clara University

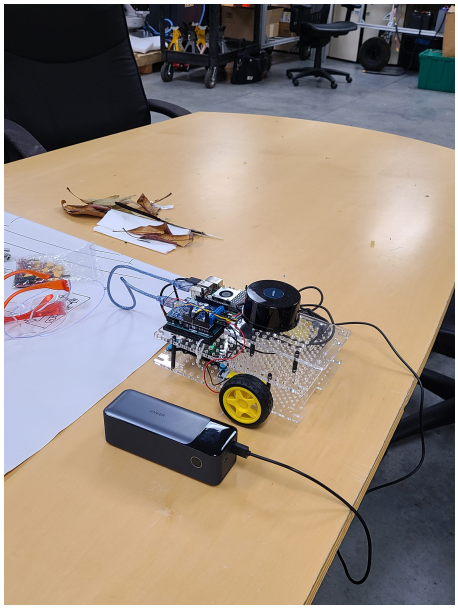


Figure 1. Physical AGV prototype used for manually pushed mapping tests. Portable compute and sensing were developed before drivetrain commissioning.

Engineering objective

Develop a complete 2D localization and mapping pipeline without relying on wheel encoders. The system estimates motion from consecutive LiDAR scans, incorporates IMU heading information, and creates an occupancy-grid map through ROS 2 integration. This enabled mapping development while the motor system remained inactive.

My implementation and integration work

Developed the Arduino-to-ROS 2 IMU interface; configured laser odometry, EKF fusion, static transforms, and asynchronous SLAM; integrated launch files and RViz; and established a stationary-data covariance workflow and SSH-based operation. Existing YDLIDAR, RF2O, robot_localization, SLAM Toolbox, and Adafruit components provide the underlying drivers and algorithms.

Demonstrated scope
Indoor mapping with manual platform movement, filtered odometry, sensor visualization, and an integrated launch workflow. Powered driving, wheel feedback, and autonomous goal navigation are subsequent development stages.

1 / Hardware and physical integration

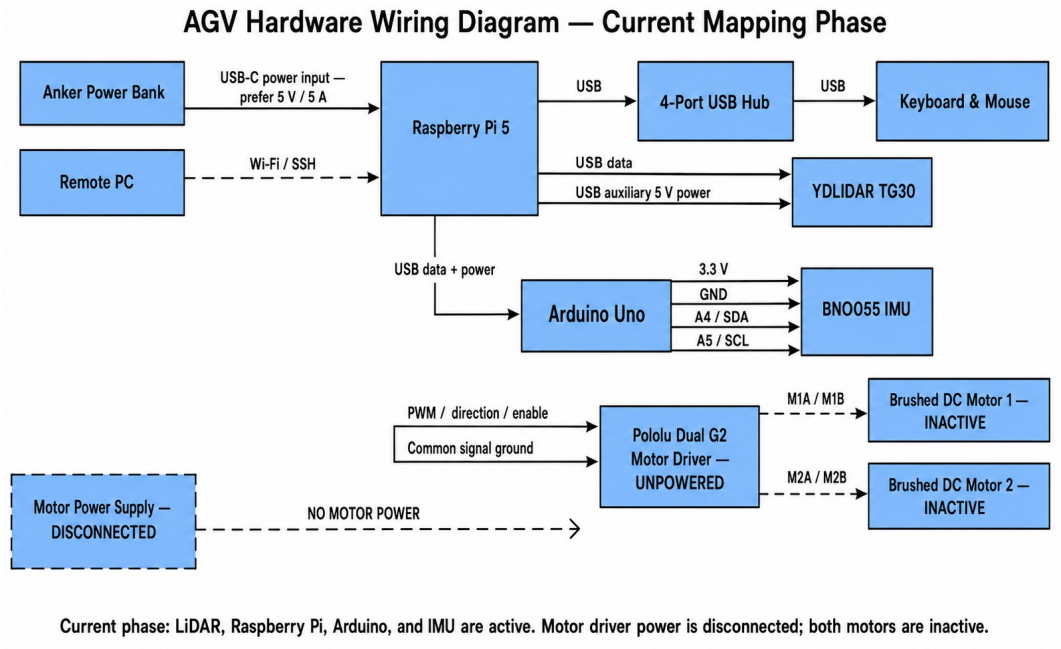


Figure 2. Supplied hardware wiring overview for the current mapping phase. Dashed motor branches represent inactive or disconnected actuation paths.

Component	Role in the mapping prototype
Raspberry Pi 5	Runs ROS 2 Jazzy, sensor interfaces, RF2O, EKF, SLAM, and visualization.
Anker power bank / USB hub	Portable compute power; hub supports keyboard and mouse peripherals.
YDLIDAR TG30	USB-connected planar scanner; /dev/ttyUSB0 in the supplied configuration.
Arduino Uno + BNO055	I²C sensor acquisition and USB serial transport; /dev/ttyACM0 at 115200 baud.
Pololu Dual G2 + DC motors	Present on the vehicle but unpowered and not controlled during documented mapping tests.

Physical integration decisions

The active subsystem consists of compute, LiDAR, microcontroller, and IMU. Separating mapping development from drivetrain commissioning made it possible to test sensing and estimation independently. The platform was moved slowly by hand to collect scans and revisit previously observed indoor structures.

The sensor frames must describe the actual mounting geometry. The submitted IMU transform is coincident with base_link, while the LiDAR launch supplies a provisional offset and rotation. Measuring and validating those extrinsics remains an important step before quantitative localization testing.

2 / High-level software architecture

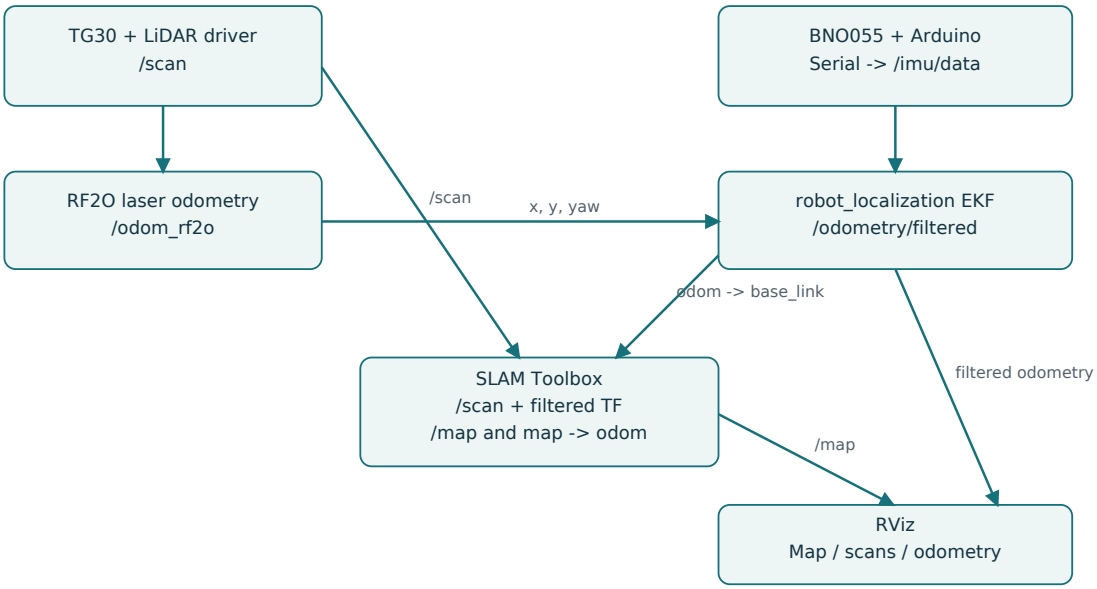


Figure 3. Software data flow reconstructed from the submitted master launch, sensor interfaces, and estimator configurations. The LiDAR stream branches to both RF2O and SLAM.

RF2O converts successive laser scans into a local motion estimate. The custom IMU node supplies quaternion orientation and angular velocity. The EKF fuses selected variables and publishes both filtered odometry and the odom-to-base_link transform. SLAM Toolbox consumes /scan and the transform chain, rather than directly subscribing to /odometry/filtered.

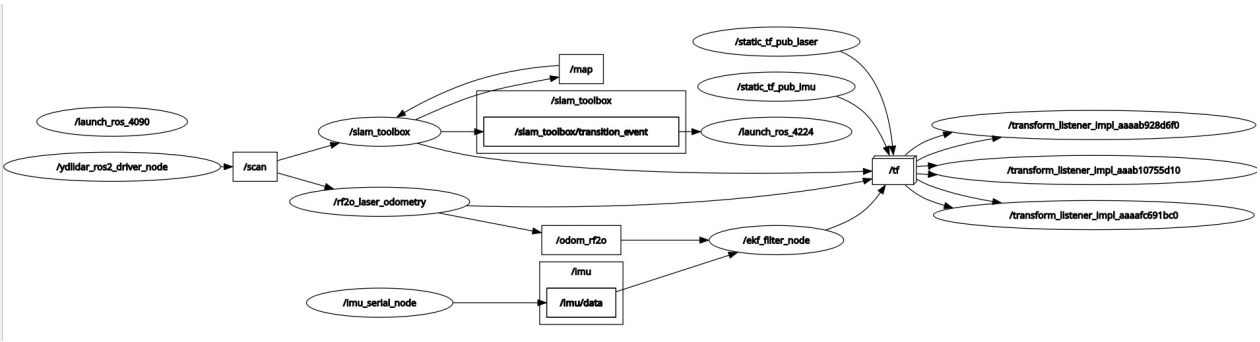


Figure 4. Cropped supplied rqt_graph capture from a development session. It shows the scan branch, IMU path, EKF, mapping node, and TF connections; the submitted launch files define the configuration described here.

Package responsibilities

Package	Integration responsibility
bno055_imu	Custom serial parsing and standard ROS sensor publications.
agv_localization	Master launch, EKF/SLAM settings, IMU static TF, and saved RViz configuration.
RF2O / YDLIDAR	Laser acquisition and odometry, with project-specific parameters and RF2O covariance assignments.
robot_localization / SLAM Toolbox	Existing estimation and mapping algorithms configured for the platform.

3 / Custom IMU acquisition and serial interface

The BNO055 is read through the Adafruit library on an Arduino Uno at I²C address 0x28. The sketch enables the external crystal when available, reports initialization status, and emits one newline-terminated CSV record per loop. Its nominal delay is 10 ms; sensor reads and serial transmission add to that loop time.

Packet fields	Contents / purpose
0-2	IMU header, sequence number, and Arduino millis timestamp.
3-6	Quaternion orientation in w, x, y, z order.
7-9	Gyroscope angular velocity in rad/s.
10-18	Raw acceleration, gravity-removed acceleration, and gravity vector.
19-22	Magnetic field in microtesla and temperature in degrees Celsius.
23-26	System, gyroscope, accelerometer, and magnetometer calibration states.

ROS node behavior

imu_serial_node validates the IMU header and exactly 27 fields, converts numeric values, and rejects malformed records. It handles IMU_READY and ERROR status lines separately. Serial faults trigger disconnection and later reconnection attempts, with a configured 2-second retry interval; opening the port also allows a 2-second reset wait.

Topic	ROS message / content
/imu/data	sensor_msgs/Imu: quaternion, angular rate, and gravity-removed linear acceleration.
/imu/mag	sensor_msgs/MagneticField: magnetic values converted from microtesla to tesla.
/imu/temperature	sensor_msgs/Temperature: sensor temperature.
/imu/calibration	std_msgs/UInt8MultiArray: four calibration status values.

Timing and interface boundaries

The Python timer is configured for 100 Hz, but publications depend on serial arrival and callback execution. The node timestamps messages using ROS time at publication; Arduino time and sequence are parsed but are not used for synchronized sensor-time stamping or packet-loss accounting. The calibration stream is published for monitoring, while the current node does not gate EKF input on calibration status.

Implementation value
The interface exposes embedded data through standard ROS messages, allowing estimation, recording, and visualization to share the same sensor stream without device-specific parsing in downstream nodes.

4 / Stationary recording and covariance treatment

A motors-off rosbag provides a recorded basis for examining IMU noise and calibration state. The covariance utility reads rosbag2 messages, supports startup/end trimming, normalizes quaternions, converts them to roll/pitch/yaw, unwraps roll and yaw, rejects non-finite data, and calculates sample covariance. Optional settings retain only the diagonal and apply a multiplicative safety factor.

Recorded / derived quantity	Value
Full recording duration	287.229 s from bag metadata.
/imu/data messages	8,610; approximately 29.98 messages/s over the full recording.
Covariance-report selection	257.229 s with 7,711 valid IMU samples; 0 invalid IMU messages reported.
Calibration order	System / gyro / accelerometer / magnetometer.
Selected calibration min and max	[0, 3, 0, 0] for both; the selection was not fully calibrated.

Published initial covariance values

Measurement	Published diagonal	Units of variance
Orientation	0.01, 0.01, 0.02	rad^2
Angular velocity	0.02, 0.02, 0.02	$(rad/s)^2$
Linear acceleration	0.04, 0.04, 0.04	$(m/s^2)^2$
Magnetic field	0.05, 0.05, 0.05	T^2

These hardcoded values are initial tuning choices, not measured sensor accuracy. The magnetic covariance is published on a separate topic and does not enter the current EKF. The 100 Hz configuration and approximately 30 Hz recorded delivery describe different quantities.

How the supplied covariance report should be interpreted

The file motors_off_covariance_safe.yaml distinguishes raw sample covariance from a 3× diagonal recommendation. For example, measured gyro variances are approximately [1.79e-6, 3.58e-6, 1.45e-6]; the recommended values are approximately [5.36e-6, 1.07e-5, 4.36e-6]. Those recommended values also appear in source comments, while the runtime still publishes the larger values above.

The utility computes Euler-angle covariance from normalized quaternion measurements. Its almost-zero stationary orientation variance indicates nearly constant reported orientation in this selection; it does not establish near-perfect heading accuracy, particularly with the recorded calibration state. Moving and powered-motor datasets are needed for representative tuning.

5 / EKF state selection and transform ownership

robot_localization runs in two-dimensional mode with world_frame set to odom. The EKF update setting is 15 Hz and sensor_timeout is 0.2 s. Control-input prediction is disabled. The state-selection arrays choose which measurements enter fusion; they do not limit which fields the source messages contain.

Input	Fused variables	Configuration
/odom_rf2o	x, y, yaw	differential: false; relative: false.
/imu/data	yaw, yaw rate	differential: false; relative: true.
IMU acceleration	Not fused	Published by the interface; all acceleration selections remain false.
RF2O velocity	Not fused	Velocity exists in odometry messages but is disabled in odom0_config.

Why these states were selected

LiDAR odometry supplies translation and heading without wheel encoders. IMU orientation contributes yaw, and the gyroscope contributes yaw rate. Setting imu0_relative to true references IMU orientation to its initial reading. Gravity-removed acceleration is available but excluded from fusion while bias, calibration, and drift remain under evaluation.

The RF2O source explicitly assigns pose variances of 0.01 for x and y and 0.03 for yaw, with large values on unused spatial axes. It also publishes twist covariance, although twist is not selected by this EKF. These are configured uncertainty assumptions rather than measured tracking errors.

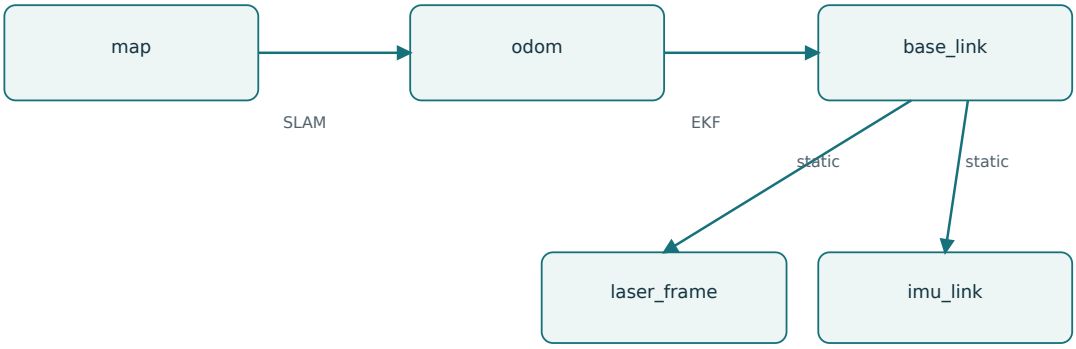


Figure 5. Frame ownership: SLAM Toolbox publishes map to odom; EKF publishes odom to base_link; static publishers connect sensor frames. RF2O TF is disabled in the master-launched configuration.

Submitted sensor transforms

base_link to imu_link uses zero translation and rotation. The LiDAR launch uses translation [-0.15, -0.05, 0.15] m and quaternion [0, 0, 1, 0] in x/y/z/w order. These describe the submitted configuration and require physical verification. The alternate rf2o.launch.py enables RF2O TF; the master instead includes rf2o_laser_odometry.launch.py with publish_tf set to false, preventing duplicate odometry transform publishers.

6 / Mapping with SLAM Toolbox

The mapping node is async_slam_toolbox_node. Its launch file explicitly requests lifecycle configuration and activates the node after it reaches the inactive state. Mapping uses /scan and the map / odom / base_link frame relationship, with scan matching and loop closure enabled.

Setting	Configured value	Purpose
Map resolution	0.05 m/cell	Occupancy-grid spacing; not a localization accuracy claim.
Laser range limits	0.10-10.0 m	Processing/rastering limits for the mapping setup.
Motion thresholds	0.10 m; 0.10 rad	Allow updates during slow manual translation and turns.
Map update interval	2.0 s	Configured grid publication/update interval.
TF publish period	0.02 s	Configured map-to-odom publication period.
Minimum time interval	0.1 s	Configured scan-processing timing threshold.
Loop search distance	3.0 m	Candidate loop-search region in this configuration.

Local motion versus global map correction

The EKF provides the local odometry transform. SLAM Toolbox relates scans to the evolving map and can adjust map-to-odom when revisited geometry supports loop closure. This division lets the local odometry frame remain continuous while global map alignment is corrected. Enabling loop closure is a configuration fact; its accuracy benefit has not been quantified in a controlled comparison.

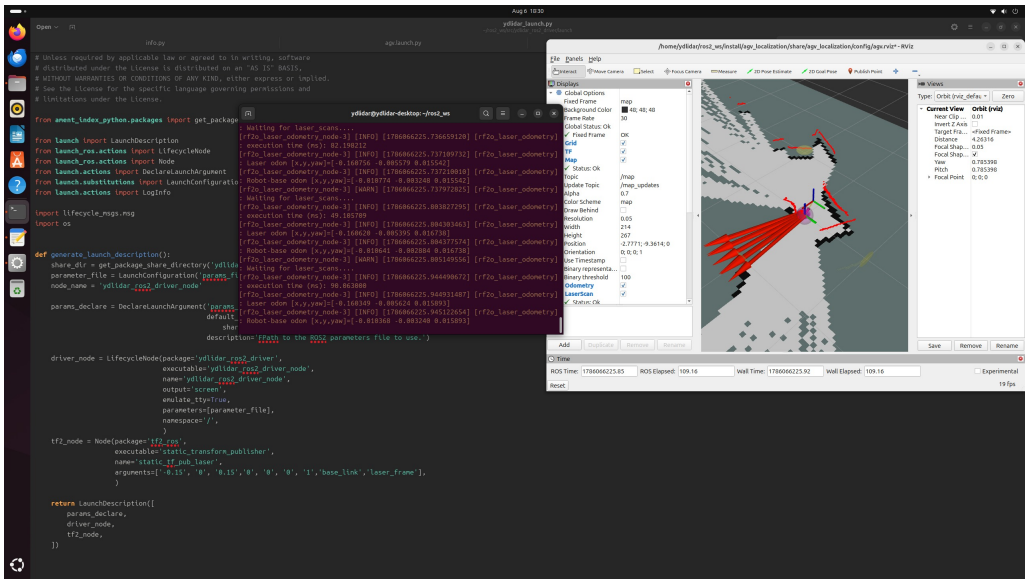


Figure 6. Supplied mapping-session screenshot with an occupancy grid and laser overlay in RViz alongside running ROS processes. The image demonstrates visual output, not a ground-truth accuracy measurement.

7 / Launch, remote operation, and map saving

Integrated startup

agv.launch.py includes LiDAR, RF2O, the IMU node, IMU static transform, EKF, SLAM, and optional RViz. It supplies a saved RViz configuration and accepts use_rviz and rviz_config arguments. The components are launched together; the file is not a sensor-readiness orchestration system, so runtime checks remain part of the workflow.

```
cd ~/ros2_ws
colcon build --symlink-install
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch agv_localization agv.launch.py
```

Before launch, upload the Arduino sketch, install the required sensor/ROS dependencies, and confirm serial device assignments and permissions. The saved configuration uses map as the RViz fixed frame and displays scans, maps, transforms, and filtered odometry.

Remote operation using SSH

The supplied instructions describe launching the Pi over SSH on the same Wi-Fi network. A headless launch avoids starting RViz on the Pi display. An alternative uses X11 forwarding so the RViz process running on the Pi appears on the remote computer; that arrangement differs from running a separate local RViz ROS node on the computer.

```
ssh username@pi_address
source /opt/ros/jazzy/setup.bash
source ~/ros2_ws/install/setup.bash
ros2 launch agv_localization agv.launch.py use_rviz:=false
```

Runtime checks and map export

```
ros2 node list
ros2 lifecycle get /slam_toolbox
ros2 topic hz /scan
ros2 topic hz /imu/data
ros2 topic hz /odometry/filtered
ros2 run tf2_ros tf2_echo odom base_link
ros2 run tf2_ros tf2_echo map laser_frame
```

Move the unpowered vehicle slowly, observe the map and scan alignment, and revisit known regions. Once /map is available, the documented map-saving path uses nav2_map_server. The map folder must exist and the relevant package must be installed.

```
mkdir -p ~/ros2_ws/maps
ros2 run nav2_map_server map_saver_cli \
  -f ~/ros2_ws/maps/my_map
```

8 / Mapping screenshots and observed outputs

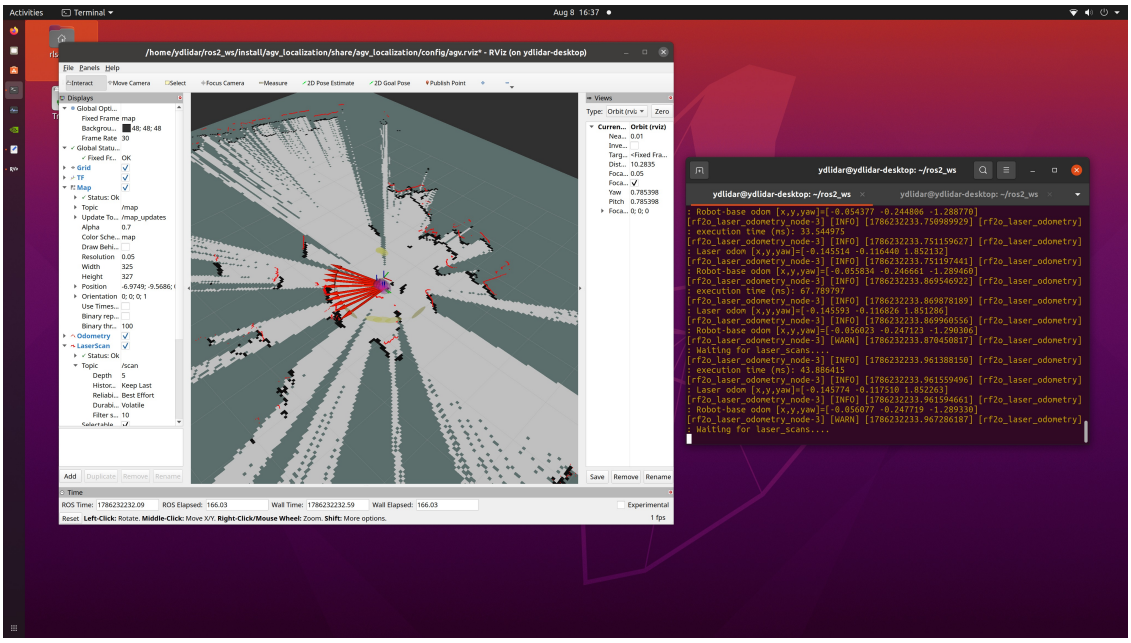


Figure 7. Full supplied desktop capture from a mapping session. RViz displays the evolving occupancy grid and current laser returns alongside terminal activity.

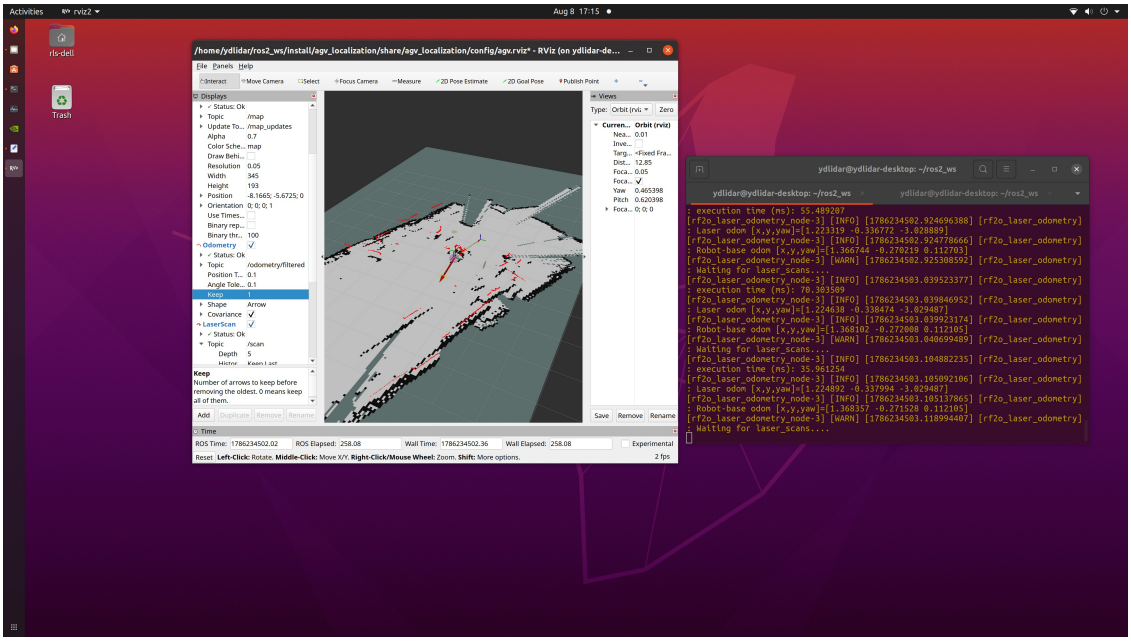


Figure 8. Another supplied mapping view showing elongated indoor geometry with scan overlays. Different views illustrate the development sessions; they are not a controlled performance comparison.

What these outputs establish

The screenshots and project documentation show integrated occupancy-grid visualization during manually pushed tests. Together with the supplied ROS interfaces and launch configuration, they support the demonstrated sensing-to-mapping workflow. Filtered odometry is available for display and diagnostic monitoring.

No ground-truth trajectory or controlled map-error benchmark accompanies these images. Grid resolution, filter frequency, and a visually recognizable map should therefore be reported as configuration and demonstration evidence rather than numerical accuracy results.

9 / Engineering decisions and diagnostic workflow

Challenge	Implementation / diagnostic response
No encoder odometry	Use RF2O scan-derived motion and fuse selected IMU heading information.
Embedded sensor integration	Use a structured serial packet and a dedicated ROS interface rather than embedding parsing in the estimator.
Transient serial failures	Reconnect after exceptions, distinguish status messages from data, and reject malformed packets.
Competing TF publishers	Disable RF2O TF in the master-launched file; give odom-to-base_link ownership to the EKF.
Estimator confidence	Keep runtime covariance explicit, inspect stationary recordings, and separate measured covariance from tuning assumptions.
Slow manual motion	Use reduced SLAM travel thresholds and monitor scan/map updates in RViz.
Remote monitoring	Provide SSH launch, optional RViz, lifecycle checks, topic-rate checks, and map export.

A practical troubleshooting sequence

1. Verify acquisition. Check serial ports and topic rates before assessing the filter. Confirm /scan contains current data and /imu/data has the expected frame and units.

2. Verify frames. Check the static sensor frames, odom-to-base_link, and map-to-odom chain. A missing or conflicting transform can look like an estimator or visualization failure.

3. Verify RViz transport settings. The project instructions recommend Best Effort, Volatile LaserScan settings when needed. Match the displayed topic and publisher QoS, then check fixed-frame and transform availability.

4. Verify lifecycle and mapping. Confirm SLAM Toolbox is active and /map is updating. Evaluate slow straight movement and turns separately, then revisit previously observed geometry.

5. Record comparable runs. Change one estimator or mapping parameter at a time and record the same route. Topic rates, transform consistency, and map appearance help diagnose behavior; repeatable reference measurements are needed to establish improvement.

10 / Outcomes, limitations, and next development

Demonstrated engineering outcome

The project integrates an encoderless 2D mapping stack on a Raspberry Pi 5, combining a custom embedded IMU interface, RF2O laser odometry, selected-state EKF fusion, SLAM Toolbox, and RViz. It enables mapping before wheel-encoder and drivetrain commissioning and provides a foundation for future wheel-encoder fusion and autonomous navigation using Nav2.

Current development boundaries

Tests used manual movement with motors inactive. Sensor mounting transforms remain provisional. The stationary dataset was not fully calibrated, and its motors-off noise does not represent powered vibration or magnetic disturbance. Source timestamps use ROS publication time, so serial delay and acquisition timing require further characterization. Quantitative trajectory accuracy and loop-closure improvement remain to be measured.

Next validation and integration steps

Priority	Work package	Evidence to collect
1	Measure LiDAR/IMU extrinsics and confirm axis alignment.	Static TF measurements and repeatable rotation checks.
2	Repeat calibrated stationary and moving IMU recordings.	Covariance, bias, warm-up, calibration state, and delivered-rate statistics.
3	Compare RF2O-only, yaw-rate-only, and yaw-plus-yaw-rate fusion.	Matched-route drift, map consistency, turn recovery, and reference trajectory error.
4	Commission motor supply/control and add wheel odometry.	Controlled speed/turn tests and powered sensor-noise recordings.
5	Integrate Nav2 after stable localization and actuation.	Goal-navigation tests with robot footprint, obstacle layers, limits, and stop behavior.

Skills demonstrated

ROS 2 integration; Python and Arduino/C++ interfacing; serial protocol design; sensor-message publishing; transform management; estimator configuration; covariance analysis; LiDAR-based localization; occupancy-grid mapping; diagnostic visualization; and technical documentation.

Supporting project materials and attribution

Supporting materials include the project reports, hardware photographs, wiring diagram, development screenshots, Arduino sketch, bno055_imu and agv_localization packages, configured RF2O and YDLIDAR code, launching instructions, MCAP recording metadata, and covariance-analysis script/report. The latest resume provides the professional context. RF2O, YDLIDAR, robot_localization, SLAM Toolbox, and Adafruit libraries are existing components; project contributions center on implementation, configuration, adaptation, and system integration.

This expanded description is based on the supplied artifacts and code inspection. No new physical tests or ROS execution were performed to generate additional performance results.